

1. Что такое ORM и почему SQLAlchemy считается одним из самых мощных инструментов ORM в Python? ORM (Object-Relational Mapping) — это технология, которая позволяет отображать структуры базы данных (таблицы, связи) на объекты объектно-ориентированного языка программирования. Общие принципы:

- Абстракция: Разработчик работает с классами и методами вместо написания SQL-запросов вручную.
- Инкапсуляция: Логика работы с данными скрыта внутри моделей.
- Автоматизация: ORM сама заботится о преобразовании типов данных между БД и Python.

Почему SQLAlchemy:

- Двухуровневая архитектура: Она состоит из Core (гибкое управление SQL) и ORM (высокоуровневая работа с объектами).
- Data Mapper: В отличие от других ORM (например, Django ORM, использующей Active Record), SQLAlchemy отделяет структуру базы от бизнес-логики, что дает больше контроля над сложными схемами.
- Поддержка множества СУБД: Единый API для PostgreSQL, MySQL, SQLite, Oracle и др.
- Мощная система связей: Позволяет детально настраивать загрузку данных (Lazy loading, Eager loading).

2. Как создать SQLAlchemy engine для подключения к различным типам баз данных?

Engine — это главный интерфейс для взаимодействия с БД, который управляет пулом соединений (Connection Pool) и диалектами (спецификой конкретной СУБД). Для создания используется функция `create_engine()`, принимающая строку подключения (URL). Примеры:

```
from sqlalchemy import create_engine
```

```
# SQLite: используется файл (три слеша для относительного пути, четыре для абсолютного)
engine_sqlite = create_engine('sqlite:///users.db')
```

```
# PostgreSQL: схема 'postgresql://логин:пароль@хост:порт/имя_бд'
# Требуется драйвер psycopg2 или asyncpg
engine_pg = create_engine('postgresql+psycopg2://admin:12345@localhost:5432/my_db')
```

```
# MySQL: схема 'mysql+driver://user:password@host/dbname'
# Требуется драйвер pymysql или mysql-connector
engine_mysql = create_engine('mysql+pymysql://root:pass@127.0.0.1:3306/store_db')
```

3. Что представляет собой объект Session в SQLAlchemy и как его правильно использовать? Session — это «рабочее пространство» или «единица работы» (Unit of Work), которая хранит в памяти все изменения объектов до тех пор, пока они не будут зафиксированы в БД. Принципы управления:

- Создание: Обычно создается через фабрику sessionmaker(bind=engine).
- Identity Map: Сессия гарантирует, что для одной строки в БД в памяти будет существовать только один уникальный Python-объект.
- Транзакционность:
 - session.add(obj): Помещает объект в состояние "pending" (ожидание).
 - session.commit(): Фиксирует транзакцию, отправляя все изменения в БД.
 - session.rollback(): Откатывает текущую транзакцию при возникновении ошибки.
 - session.close(): Закрывает сессию и возвращает соединение в пул. Пример использования:

```
from sqlalchemy.orm import sessionmaker
Session = sessionmaker(bind=engine)
session = Session()
```

```
try:
    user = User(name="Ivan")
    session.add(user)
    session.commit() # Сохранение
```

except Exception:

```
    session.rollback() # Откат при ошибке
```

finally:

```
    session.close() # Обязательное закрытие
```

4. Какая роль у Declarative Base в SQLAlchemy? Declarative Base — это базовый класс для всех моделей ORM. Он объединяет три элемента: таблицу (Table), класс Python и отображение (Mapper) между ними. Как это работает: Когда мы наследуемся от него, SQLAlchemy автоматически регистрирует класс в своих метаданных, связывая его с конкретной таблицей. Пример:

```
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
```

```
# Современный стиль (2.0+)
```

```
class Base(DeclarativeBase):
```

```
    pass
```

```
class Product(Base):
```

```
    __tablename__ = "products" # Имя таблицы в БД
```

```
    id: Mapped[int] = mapped_column(primary_key=True)
```

```
    name: Mapped[str]
```

```
    price: Mapped[float]
```

5. Чем отличается подход SQLAlchemy Core от ORM-подхода?

- SQLAlchemy Core:

- Ориентирован на схему данных (таблицы, столбцы).

- Использует SQL Expression Language (построение запросов через функции `select()`, `insert()`).

- Возвращает простые структуры (Row).

- Когда использовать: Сложные аналитические запросы, высокая нагрузка, где ORM-объекты слишком тяжелы.

- SQLAlchemy ORM:

- Ориентирован на доменные модели (объекты классов).
- Абстрагирует работу с БД до уровня методов объектов.
- Возвращает экземпляры классов моделей.
- Когда использовать: Бизнес-логика, CRUD операции, веб-приложения со сложными связями.

6. Как задать отношения между таблицами (1:N, M:N) в SQLAlchemy? Отношения задаются через ForeignKey (на уровне БД) и relationship (на уровне ORM).

- Один-ко-многим (One-to-Many):

```
class Parent(Base):
    __tablename__ = "parent"
    id = mapped_column(Integer, primary_key=True)
    # back_populates синхронизирует изменения в обоих классах
    children = relationship("Child", back_populates="parent")
```

```
class Child(Base):
    __tablename__ = "child"
    id = mapped_column(Integer, primary_key=True)
    parent_id = mapped_column(ForeignKey("parent.id"))
    parent = relationship("Parent", back_populates="children")
```

- Многие-ко-многим (Many-to-Many): Требуется промежуточную таблицу (Association Table).

```
# Таблица связи
user_group = Table("user_group", Base.metadata,
    Column("user_id", ForeignKey("users.id")),
    Column("group_id", ForeignKey("groups.id"))
)
```

```
class User(Base):
```

```
__tablename__ = "users"
id = mapped_column(Integer, primary_key=True)
groups = relationship("Group", secondary=user_group)
```

7. Как выполнять операции CRUD с использованием SQLAlchemy ORM? CRUD — это базовый набор операций с данными:

- Create (Создание):

```
new_item = Item(name="Phone")
session.add(new_item)
session.commit()
```

- Read (Чтение):

```
# Получить по ID
item = session.get(Item, 1)
# Фильтрация
items = session.query(Item).filter(Item.price > 100).all()
```

- Update (Обновление):

```
item = session.get(Item, 1)
item.price = 200 # Просто меняем атрибут
session.commit()
```

- Delete (Удаление):

```
item = session.get(Item, 1)
session.delete(item)
session.commit()
```

8. Фильтрация, сортировка и объединение (join) в SQLAlchemy Для построения запросов в SQLAlchemy (версии 2.0+) используются функции select,

filter, order_by и join.

- Фильтрация: filter_by() используется для простых равенств, filter() — для сложных условий (с операторами >, <, !=).
 - Сортировка: Метод order_by() принимает столбцы модели.
 - Объединение (Join): Метод join() связывает таблицы по внешнему ключу.
- Пример:

```
from sqlalchemy import select
```

```
# Выбрать пользователей с именем 'Ivan', отсортированных по ID, включая их посты
```

```
stmt = (  
    select(User)  
    .join(Post) # Объединение таблиц  
    .filter(User.name == "Ivan") # Фильтрация  
    .order_by(User.id.desc()) # Сортировка по убыванию  
)  
results = session.execute(stmt).scalars().all()
```

9. Выполнение «сырых» SQL-запросов Если возможностей ORM недостаточно, можно использовать функцию text() для выполнения прямого SQL-кода. Это полезно для специфических функций БД или оптимизации сложных запросов. Пример:

```
from sqlalchemy import text
```

```
# Выполнение сырого запроса с параметрами (защита от SQL-инъекций)
```

```
query = text("SELECT * FROM users WHERE status = :status")
```

```
result = session.execute(query, {"status": "active"})
```

```
for row in result:
```

```
    print(row.name)
```

10. Управление транзакциями и обработка исключений Транзакция — это группа операций, которая либо выполняется полностью, либо не выполняется

вовсе.

- Контекстный менеджер: В современных версиях рекомендуется использовать `session.begin()`, который автоматически сделает `commit` при успехе или `rollback` при ошибке.
- Ручное управление: Используется блок `try-except`. Пример:

Автоматическое управление транзакцией

with `session.begin()`:

```
user = User(name="Alex")
```

```
session.add(user)
```

`commit` произойдет автоматически в конце блока, если нет ошибок

11. Система пула соединений (Connection Pool) Пул соединений — это набор уже открытых соединений с БД, которые поддерживаются в активном состоянии.

- Зачем нужен: Создание нового соединения с БД — дорогая операция (занимает время). Пул позволяет «переиспользовать» старые соединения.
- Влияние на производительность: При высокой нагрузке пул предотвращает падение БД от слишком большого количества запросов и ускоряет ответ приложения, так как соединение уже готово.
- Настройка: Задается при создании `engine` (параметры `pool_size`, `max_overflow`).

12. Стратегии загрузки (Lazy vs Eager loading) Это способы получения связанных данных (например, постов пользователя).

- Lazy Loading (Ленивая): Связанные данные подгружаются отдельным запросом только в момент обращения к ним.
 - Плюс: Экономия памяти. Минус: Проблема N+1 (много мелких запросов к БД).
- Eager Loading (Жадная): Связанные данные загружаются сразу одним запросом вместе с основным объектом (через `JOIN`).
 - Плюс: Минимум запросов к БД. Минус: Запрос может стать очень тяжелым.
- Когда использовать: Eager — если точно знаете, что данные понадобятся. Lazy — если связанные данные нужны редко.

13. Работа с сессией в веб-приложениях Главный принцип: одна сессия на один HTTP-запрос.

- Жизненный цикл: Сессия создается в начале обработки запроса и закрывается (вместе с коммитом или откатом) в самом конце.
- Интеграция: В Flask обычно используют расширение Flask-SQLAlchemy, которое само управляет сессиями.
- Scoped Session: Специальный объект `scoped_session` гарантирует, что каждый поток или запрос получит свою изолированную сессию, предотвращая утечки данных между пользователями.

14. Оптимизация запросов и производительность

- Индексы: Обязательно создавать индексы в БД для столбцов, по которым идет частый поиск.
- Пагинация: Использовать `limit()` и `offset()`, чтобы не загружать миллионы строк за раз.
- Профилирование: Включение `echo=True` в `engine` позволяет видеть все SQL-запросы в консоли для анализа их эффективности.
- Bulk-операции: Для вставки тысяч строк использовать `session.bulk_insert_mappings()` вместо добавления объектов по одному.
- Кэширование: Использование Redis для хранения результатов часто повторяющихся запросов.

15. Что такое Flask и почему он «микро»? Flask — это легковесный веб-фреймворк на Python для разработки веб-приложений.

- Почему микрофреймворк: Он не включает в себя «из коробки» инструменты для работы с БД, валидацию форм или систему авторизации. Он предоставляет только ядро (маршрутизация, обработка запросов, шаблонизатор Jinja2).
- Философия: Предоставить разработчику свободу выбора библиотек (например, можно выбрать SQLAlchemy или Tortoise ORM).
- Области применения: Микросервисы, REST API, небольшие и средние проекты,

прототипирование.

16. Базовое Flask-приложение и настройка маршрутизации Маршрутизация во Flask

связывает URL-адрес с конкретной функцией (view-function), которая обрабатывает запрос. Это реализуется с помощью декоратора `@app.route()`. Пример:

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
# Главная страница
```

```
@app.route('/')
```

```
def index():
```

```
    return "Главная страница"
```

```
# Динамический маршрут с параметром
```

```
@app.route('/user/<name>')
```

```
def show_user(name):
```

```
    return f"Привет, {name}!"
```

```
if __name__ == '__main__':
```

```
    app.run()
```

17. Обработка GET и POST запросов в Flask По умолчанию маршруты принимают только

GET-запросы. Для обработки отправки данных (например, форм) нужно явно указать методы в параметре `methods`.

- GET: Используется для получения данных.

- POST: Используется для отправки данных на сервер. Пример:

```
from flask import request
```

```
@app.route('/login', methods=['GET', 'POST'])
```

```
def login():
    if request.method == 'POST':
        username = request.form.get('username')
        return f"Вы вошли как {username}"
    return "<form method='post'><input name='username'><button>OK</button></form>"
```

18. Jinja2 и использование шаблонизатора Jinja2 — это мощный движок шаблонов, который позволяет отделять HTML-код от логики Python. Flask ищет шаблоны в папке templates/. Возможности:

- Переменные: Выводятся в двойных фигурных скобках: {{ username }}.
- Управляющие конструкции: Пишутся внутри {% ... %} (циклы for, условия if).
- Наследование: Позволяет создать базовый скелет сайта и расширять его в других шаблонах. Пример шаблона:

```
<ul>
{% for item in items %}
    <li>{{ item.name }}</li>
{% endfor %}
</ul>
```

19. Обработка форм и валидация (Flask-WTF) Для безопасной работы с формами используется расширение Flask-WTF. Оно защищает от CSRF-атак и упрощает валидацию. Принцип работы:

20. Создается класс формы, наследуемый от FlaskForm.

21. В методе маршрута проверяется form.validate_on_submit(). Пример:

```
from flask_wtf import FlaskForm
from wtforms import StringField
from wtforms.validators import DataRequired

class NameForm(FlaskForm):
    name = StringField('Имя', validators=[DataRequired()])
```

```
@app.route('/form', methods=['GET', 'POST'])
def handle_form():
    form = NameForm()
    if form.validate_on_submit():
        return f"Данные верны: {form.name.data}"
    return render_template('form.html', form=form)
```

20. Обработка ошибок и кастомные страницы ошибок Flask позволяет перехватывать ошибки (например, 404 Not Found или 500 Internal Server Error) с помощью декоратора `@app.errorhandler()`. Пример:

```
@app.errorhandler(404)
def page_not_found(e):
    # Возвращаем свой шаблон и код ошибки
    return render_template('404.html'), 404
```

```
@app.errorhandler(500)
def server_error(e):
    return "Ошибка сервера", 500
```

21. Роль Flask-расширений и их интеграция Поскольку Flask — микрофреймворк, дополнительная функциональность добавляется через расширения. Основные расширения:

- Flask-SQLAlchemy: Работа с базами данных.
- Flask-Login: Управление сессиями пользователей и авторизацией.
- Flask-Migrate: Управление миграциями БД. Интеграция: Обычно расширение инициализируется либо сразу, либо через метод `init_app(app)`.

```
from flask_sqlalchemy import SQLAlchemy
```

```
db = SQLAlchemy()
```

```
def create_app():
```

```
app = Flask(__name__)
db.init_app(app) # Подключение расширения к приложению
return app
```

22. Система сессий во Flask Сессия позволяет хранить информацию о пользователе между разными запросами.

- Механизм: Flask использует клиентские сессии. Данные шифруются и хранятся в Cookie в браузере пользователя.
- Защита: Для работы сессий обязательно нужно задать `app.secret_key`. Без него данные можно прочитать, но нельзя подделать.
- Пример использования:

```
from flask import session
```

```
app.secret_key = 'super-secret-key'
```

```
@app.route('/set')
```

```
def set_session():
```

```
    session['user_id'] = 123
```

```
    return "ID сохранен в сессии"
```

```
@app.route('/get')
```

```
def get_session():
```

```
    user_id = session.get('user_id')
```

```
    return f"Ваш ID из сессии: {user_id}"
```

23. Работа со статическими файлами во Flask Flask автоматически ищет статические файлы (CSS, JS, картинки) в папке `static`, находящейся в корне приложения.

- Организация: Обычно внутри `static` создают подпапки `css/`, `js/`, `img/`.
- Вызов в шаблонах: Для формирования путей используется функция `url_for`. Это гарантирует правильность пути при переносе приложения. Пример в HTML:

```
<link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">

```

24. Создание RESTful API во Flask REST API во Flask строится на обмене данными в формате JSON и использовании стандартных методов HTTP.

- Прием JSON: Используется `request.get_json()`.
- Возврат JSON: Используется функция `jsonify()`. Пример:

```
from flask import jsonify, request
```

```
@app.route('/api/data', methods=['POST'])
def process_api():
    data = request.get_json() # Получаем JSON из запроса
    name = data.get('name', 'Guest')
    return jsonify({"message": f"Hello, {name}", "status": "success"}), 201
```

25. Аутентификация и авторизация во Flask

- Flask-Login: Самое популярное решение для сессионной аутентификации. Управляет входом/выходом и предоставляет декоратор `@login_required` для защиты маршрутов.
- Flask-JWT-Extended: Используется для API. Вместо сессий выдает токен (JSON Web Token), который клиент прикрепляет к заголовкам запросов.
- Внешние сервисы: Интеграция через OAuth (Google, GitHub) обычно реализуется с помощью библиотеки Authlib или Flask-Dance.

26. Тестирование Flask-приложений Для тестирования Flask предоставляет специальный тестовый клиент, который имитирует запросы к приложению без запуска реального веб-сервера.

- Юнит-тесты: Проверка отдельных функций или моделей.
- Функциональные тесты: Проверка конкретного маршрута и логики обработки запроса. Пример:

```
def test_home_page():  
    app.config['TESTING'] = True  
    with app.test_client() as client:  
        response = client.get('/')  
        assert response.status_code == 200  
        assert b"Welcome" in response.data
```

27. Что такое Django и философия «батарейки в комплекте» Django — это высокоуровневый веб-фреймворк на Python, предназначенный для быстрой разработки сложных приложений.

- «Батарейки в комплекте»: Это философия Django, означающая, что почти всё необходимое уже встроено: админ-панель, ORM, аутентификация, миграции, защита от атак (CSRF, XSS), работа с формами.
- Отличие: В отличие от микрофреймворков (Flask), Django диктует строгую структуру проекта, что облегчает работу в больших командах.

28. Архитектура MVT в Django MVT (Model-View-Template) — это модификация классического MVC.

- Model (Модель): Описание структуры данных (классы Python). Отвечает за взаимодействие с БД через ORM.
- View (Представление): Логика обработки запроса. Получает данные из моделей и решает, какой шаблон отобразить.
- Template (Шаблон): HTML-файл с синтаксисом Django Template Language (DTL). Отвечает за то, как данные будут выглядеть.
- Взаимодействие: Запрос -> URL Conf -> View -> Model (опционально) -> Template -> Ответ.

29. Создание проекта и приложения в Django Проект в Django — это общая конфигурация сайта, а приложение — это отдельный модуль функциональности (например, "блог", "форум"). Последовательность команд:

30. Установка: `pip install django`

31. Создание проекта: `django-admin startproject myproject`

32. Переход в папку: `cd myproject`

33. Создание приложения: `python manage.py startapp myapp`

34. Запуск сервера: `python manage.py runserver`

35. Настройка маршрутов (URL Dispatcher) в Django Маршрутизация настраивается в файлах `urls.py`.

- Роль: Сопоставление URL-пути с конкретной функцией или классом представления (view).
- Принцип включения: Для чистоты кода маршруты приложения обычно выносятся в отдельный `urls.py` внутри приложения и подключаются в главный файл через `include()`.
- Параметры: Позволяют захватывать части URL. Пример:

```
from django.urls import path, include
```

```
# В основном urls.py проекта
```

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('blog/', include('blog.urls')), # Подключаем маршруты приложения  
]
```

```
# В urls.py приложения blog
```

```
urlpatterns = [  
    path("", views.index, name='index'),  
    path('post/<int:post_id>/', views.detail, name='detail'), # С числовым параметром  
]
```

31. Модели Django: определение и миграции Модель в Django — это Python-класс, который описывает структуру таблицы в базе данных.

- Синтаксис: Наследуется от `models.Model`. Каждый атрибут класса — это поле таблицы (`models.CharField`, `models.DateTimeField` и др.).

- Миграции: Это способ переноса изменений из кода моделей в структуру БД.

1. `python manage.py makemigrations` — анализирует изменения в моделях и создает файл миграции (инструкцию).

2. `python manage.py migrate` — применяет эти инструкции к базе данных.

Пример:

```
from django.db import models
```

```
class Article(models.Model):
```

```
    title = models.CharField(max_length=100)
```

```
    content = models.TextField()
```

```
    created_at = models.DateTimeField(auto_now_add=True)
```

32. Django ORM: работа и преимущества Django ORM позволяет взаимодействовать с БД с помощью методов Python вместо написания SQL.

- Преимущества:

- Безопасность: Автоматически защищает от SQL-инъекций через параметризацию запросов.

- Переносимость: Код не зависит от типа БД (можно легко перейти с SQLite на PostgreSQL).

- Автоматизация: Сама создает сложные JOIN-запросы и следит за типами данных.

- Кэширование: QuerySet-ы (результаты запросов) кэшируются внутри объекта, что предотвращает лишние обращения к БД при повторном использовании.

33. Админ-панель Django и её кастомизация Это готовый интерфейс для управления данными сайта.

- Регистрация: Чтобы модель появилась в админке, её нужно зарегистрировать в `admin.py`.
- Кастомизация: Можно настроить через классы, наследуемые от `admin.ModelAdmin`.
 - `list_display`: выбор столбцов для отображения.
 - `list_filter`: добавление фильтров сбоку.
 - `search_fields`: добавление поиска по полям. Пример:

```
@admin.register(Article)
class ArticleAdmin(admin.ModelAdmin):
    list_display = ('title', 'created_at')
```

34. Обработка форм в Django (Form и ModelForm)

- Form: Описывает поля вручную. Используется для задач, не связанных напрямую с моделями (например, форма обратной связи).
- ModelForm: Автоматически создает поля на основе существующей модели. Пример обработки в представлении (View):

```
def create_article(request):
    if request.method == 'POST':
        form = ArticleForm(request.POST)
        if form.is_valid(): # Валидация
            form.save() # Сохранение в БД
            return redirect('success')
    else:
        form = ArticleForm()
    return render(request, 'form.html', {'form': form})
```

35. Аутентификация и авторизация в Django В Django встроена мощная система пользователей (`django.contrib.auth`).

- Аутентификация: Проверка подлинности (логин/пароль). Используются функции `authenticate()` и `login()`.

- Авторизация: Проверка прав доступа.
- Механизмы:
 - Встроенная модель User (username, email, password).
 - Декоратор `@login_required` для ограничения доступа к страницам.
 - Система разрешений (Permissions) и групп пользователей.

36. Django Middleware Middleware — это слой («прослойка»), который перехватывает запрос до того, как он попадет в View, и ответ после того, как View его сформировала.

- Зачем нужно: Для глобальных задач: проверка авторизации, сжатие ответов, логирование, защита от CSRF, кэширование всего сайта.
- Принцип: Запросы проходят через список middleware по порядку сверху вниз, а ответы — снизу вверх.

37. Шаблоны Django (DTL) Язык шаблонов Django (DTL) позволяет динамически формировать HTML.

- Синтаксис:
 - `{{ variable }}` — вывод переменной.
 - `{% tag %}` — логика (циклы for, условия if).
 - `{{ value|filter }}` — фильтры для преобразования данных (например, lower, date).
- Наследование: С помощью `{% extends "base.html" %}` и `{% block content %}` можно создать общую структуру сайта и менять только содержимое.

38. FBV (на функциях) против CBV (на классах)

- Function-Based Views (FBV):
 - Плюсы: Просты в чтении, прямолинейны.
 - Минусы: Трудно переиспользовать код, часто получается громоздкий код для стандартных задач.
- Class-Based Views (CBV):
 - Плюсы: Высокая степень переиспользования кода через наследование и

примеси (Mixins). Есть встроенные классы для типичных задач (ListView, DetailView, CreateView).

- Минусы: Сложнее в понимании из-за скрытой логики за методами класса.

Применение: FBV — для уникальной/сложной логики, CBV — для стандартных операций CRUD.

39. Статические и медиа файлы в Django Django разделяет файлы на две категории:

static (ресурсы разработчика: CSS, JS) и media (файлы, загруженные пользователями).

- Настройки:

- STATIC_URL: URL-префикс для доступа к статике (напр. /static/).

- STATICFILES_DIRS: Список папок, где Django ищет статику в режиме разработки.

- MEDIA_URL / MEDIA_ROOT: URL-префикс и абсолютный путь на диске для пользовательских файлов.

- Раздача файлов:

- Разработка (DEBUG=True): Django сам раздает файлы через встроенный сервер.

- Production: Используется команда `python manage.py collectstatic`, которая собирает все файлы в одну папку (STATIC_ROOT). Раздачей этих файлов должен заниматься веб-сервер (Nginx или Apache), так как Django делает это медленно и небезопасно для продакшена.

40. Управление настройками для разных окружений Для безопасности и гибкости

настройки разделяют (dev, test, prod).

- Подходы:

1. Раздельные файлы: Создание папки settings/ с файлами base.py, local.py, prod.py.

2. Переменные окружения: Использование библиотеки python-dotenv или django-environ для хранения секретов (пароли БД, SECRET_KEY) вне кода.

- Best Practices:

- Никогда не хранить SECRET_KEY в репозитории (Git).

- В продакшене всегда `DEBUG = False`.
- Использовать разные базы данных для тестов и разработки.

41. Механизмы безопасности Django Django спроектирован так, чтобы защищать от распространенных атак «из коробки»:

- CSRF (Межсайтовая подделка запроса): Использование тега `{% csrf_token %}` в формах проверяет, что запрос пришел именно с вашего сайта.
- XSS (Межсайтовый скриптинг): Движок шаблонов Django автоматически экранирует опасные HTML-символы в переменных.
- SQL-инъекции: ORM использует параметризацию запросов, что исключает возможность подмены SQL-кода через ввод пользователя.
- Clickjacking: Middleware `XFrameOptionsMiddleware` запрещает отображать сайт во фреймах на чужих доменах.

42. Django Rest Framework (DRF) DRF — это мощное расширение для создания REST API на базе Django.

- Основные понятия:
 - Сериализаторы (Serializers): Преобразуют объекты `QuerySet` в JSON и обратно.
 - `ViewSet`'ы: Объединяют логику обработки запросов (GET, POST, PUT, DELETE) в одном классе.
 - Роутеры (Routers): Автоматически генерируют URL-адреса для API на основе `ViewSet`.
- Пример:

```
class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email']
```

- Преимущества: Удобный веб-интерфейс для тестирования API (Browsable API), поддержка OAuth2 и JWT.

43. Пагинация, фильтрация и сортировка

- Пагинация: В Django есть встроенный класс Paginator. В DRF она настраивается одной строкой в settings.py.
- Сортировка: В ORM реализуется методом .order_by('field_name').
- Фильтрация:
 - Базовая: Метод .filter(name__icontains='term').
 - Продвинутая: Использование библиотеки django-filter, которая позволяет создавать сложные фильтры через URL-параметры (например, ?price__gt=100).

44. Оптимизация производительности

- Оптимизация ORM: Использование select_related (для связи 1:1 и 1:N через SQL JOIN) и prefetch_related (для M:M через отдельный запрос), чтобы избежать проблемы N+1.
- Кэширование: Использование Redis или Memcached для хранения тяжелых страниц или фрагментов шаблонов ({% cache %}).
- База данных: Создание индексов для полей, по которым часто идет поиск.
- Фронтенд: Использование CDN для раздачи статики и медиа, сжатие изображений и минификация CSS/JS.
- Сервер: Использование Gunicorn/Uvicorn с несколькими воркерами (процессами).